CST1510 Programming for data communication and networks

Week 9: Data base implementation implementation

LAB ACTIVITY

Refer to the `Shopping System Implementation` below as an example to your database design and client-server interaction. Here are key points to consider for your implementation:

Database Design:

The shopping system utilizes SQLite3 for handling inventory and transactions. Think about how you can model your project's data (e.g., accounts, portfolios, vending machine inventory) in a similar structured database.

Design tables that reflect your project's requirements. For example:

- Crypto Investment Platform: Tables for accounts, transactions, and portfolio.
- Smart Vending Machine: Tables for products, cart, and transactions.

Client-Server Communication:

Notice how the shopping system uses socket programming to handle client requests and server responses. You can replicate this mechanism to enable client-server communication in your project.

For instance:

- Crypto Investment Platform: The server can handle buy/sell requests and portfolio updates.
- Smart Vending Machine: The server can manage inventory, process purchases, and update stock.

---

1. SQL File (`shopping_system.sql`)

This `.sql` file initializes the database schema for the shopping system.

In [ ]:
```sql
-- Create the database schema
CREATE TABLE Products (
    productID INTEGER PRIMARY KEY AUTOINCREMENT,
    productName TEXT NOT NULL,
    price REAL NOT NULL,
    stock INTEGER NOT NULL
);

CREATE TABLE Cart (
    cartID INTEGER PRIMARY KEY AUTOINCREMENT,
    productID INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    FOREIGN KEY (productID) REFERENCES Products(productID)
);

-- Insert sample data into Products table
INSERT INTO Products (productName, price, stock) VALUES
('Sneakers', 49.99, 10),
('Backpack', 29.99, 15),
('Water Bottle', 9.99, 20),
('Laptop', 799.99, 5),
('Smartphone', 599.99, 8),
('Headphones', 99.99, 25);
```

2. Server-Side Implementation (`server.py`)

In [ ]:
```python
import sqlite3
import socket

# Initialize the database from the SQL file
def initialize_database():
    conn = sqlite3.connect("shopping_system.db")
    cursor = conn.cursor()
```

```python
    with open("shopping_system.sql", "r") as sql_file:
        sql_script = sql_file.read()
    cursor.executescript(sql_script)
    conn.commit()
    conn.close()

# Handle client requests
def handle_client(client_socket):
    conn = sqlite3.connect("shopping_system.db")
    cursor = conn.cursor()

    while True:
        try:
            # Receive request from the client
            request = client_socket.recv(1024).decode()
            if not request:
                break

            # Parse the request
            command, *args = request.split("|")

            if command == "view_products":
                cursor.execute("SELECT * FROM Products")
                products = cursor.fetchall()
                response = "\n".join(
                    f"{row[0]}|{row[1]}|{row[2]}|{row[3]}" for row in products
                )
                client_socket.send(response.encode())

            elif command == "add_to_cart":
                product_id, quantity = map(int, args)
                cursor.execute("SELECT stock FROM Products WHERE productID = ?", (product_id,))
                product = cursor.fetchone()
                if product and product[0] >= quantity:
                    cursor.execute("INSERT INTO Cart (productID, quantity) VALUES (?, ?)", (product_id, quantity))
                    cursor.execute("UPDATE Products SET stock = stock - ? WHERE productID = ?", (quantity, product_
                    conn.commit()
                    client_socket.send("Added to cart".encode())
                else:
                    client_socket.send("Insufficient stock".encode())
```

```python
            elif command == "view_cart":
                cursor.execute("""
                SELECT c.cartID, p.productName, c.quantity, p.price, (c.quantity * p.price) AS totalPrice
                FROM Cart c
                JOIN Products p ON c.productID = p.productID
                """)
                cart_items = cursor.fetchall()
                response = "\n".join(
                    f"{row[0]}|{row[1]}|{row[2]}|{row[3]}|{row[4]}" for row in cart_items
                )
                client_socket.send(response.encode())

            elif command == "checkout":
                cursor.execute("""
                SELECT c.productID, p.productName, c.quantity, (c.quantity * p.price) AS totalPrice
                FROM Cart c
                JOIN Products p ON c.productID = p.productID
                """)
                cart_items = cursor.fetchall()
                if not cart_items:
                    client_socket.send("Cart is empty".encode())
                else:
                    cursor.execute("DELETE FROM Cart")
                    conn.commit()
                    response = "\n".join(
                        f"{row[1]}|{row[2]}|{row[3]}" for row in cart_items
                    )
                    client_socket.send(response.encode())

            elif command == "exit":
                client_socket.send("Goodbye!".encode())
                break
        except Exception as e:
            client_socket.send(f"Error: {str(e)}".encode())

    conn.close()
    client_socket.close()

# Start the server
def start_server():
    initialize_database()
```

```
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(("127.0.0.1", 5000))
    server_socket.listen(5)
    print("Server is running on port 5000...")

    while True:
        client_socket, addr = server_socket.accept()
        print(f"Connection from {addr}")
        handle_client(client_socket)

if __name__ == "__main__":
    start_server()
```

3. Client-Side Implementation (`client.py`)

In [ ]:

```
import socket

# Helper function to send requests to the server
def send_request(request):
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect(("127.0.0.1", 5000))
    client_socket.send(request.encode())
    response = client_socket.recv(4096).decode()
    client_socket.close()
    return response

def view_products():
    response = send_request("view_products")
    print("\nAvailable Products:")
    print("ID | Product Name       | Price   | Stock")
    print("----------------------------------------")
    for line in response.split("\n"):
        if line.strip():
            product_id, name, price, stock = line.split("|")
            print(f"{product_id:<3} | {name:<18} | ${price:<7} | {stock}")

def add_to_cart():
    product_id = input("Enter Product ID: ")
    quantity = input("Enter Quantity: ")
```

```python
    response = send_request(f"add_to_cart|{product_id}|{quantity}")
    print(response)

def view_cart():
    response = send_request("view_cart")
    print("\nYour Cart:")
    print("ID | Product Name       | Quantity | Price   | Total")
    print("-----------------------------------------------------")
    for line in response.split("\n"):
        if line.strip():
            cart_id, name, quantity, price, total = line.split("|")
            print(f"{cart_id:<3} | {name:<18} | {quantity:<8} | ${price:<7} | ${total}")

def checkout():
    response = send_request("checkout")
    if "Cart is empty" in response:
        print("Cart is empty.")
    else:
        print("Checkout successful!")
        print("Transaction Summary:")
        for line in response.split("\n"):
            if line.strip():
                name, quantity, total = line.split("|")
                print(f"{name} - Quantity: {quantity}, Total: ${total}")

def menu():
    while True:
        print("\nMenu:")
        print("1. View Products")
        print("2. Add to Cart")
        print("3. View Cart")
        print("4. Checkout")
        print("5. Exit")
        choice = input("Enter your choice: ")
        if choice == "1":
            view_products()
        elif choice == "2":
            add_to_cart()
        elif choice == "3":
            view_cart()
        elif choice == "4":
```

```
            checkout()
        elif choice == "5":
            send_request("exit")
            print("Thank you for using the shopping system!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    menu()
```

How to Run the System

- Save the SQL file as `shopping_system.sql`
- Start the server by running `server.py` This will create the database and populate it with initial data.
- Run the client by executing `client.py`. Use the menu to interact with the shopping system.

In [ ]: